Multi-Mode CORDIC Architecture Design and Implementation Report

Setayesh Saeidi

Executive Summary

This document presents the complete design, implementation, and verification of a Multi-Mode CORDIC (COordinate Rotation DIgital Computer) algorithm supporting both **Circular (m=1)** and **Linear (m=0)** modes in Rotation Mode. The design achieves high accuracy with minimal hardware resources using only shift-and-add operations.

Key Specifications: - **Word Length:** 32 bits (Q4.28 fixed-point format) - **Iterations:** 16 (optimized for accuracy vs. latency) - **Modes:** Circular (sin/cos) and Linear (multiplication) - **Architecture:** Sequential (resource-sharing) - **Maximum Error:** $< 2\times10^{-5}$ for all operations

---

Table of Contents

1. Introduction
2. CORDIC Algorithm Background
3. Numerical Representation and Quantization
4. Accuracy Analysis
5. Hardware Architecture
6. Implementation Details
7. Verification Results
8. Synthesis and Performance
9. Conclusions
10. Appendices

---

1. Introduction

1.1 Project Objective

The objective of this project is to implement a hardware-efficient Multi-Mode CORDIC algorithm capable of: 1. Computing trigonometric functions (sin and cos) using Circular mode 2. Performing multiplication using Linear mode 3. Maintaining high precision through proper fixed-point arithmetic 4. Optimizing hardware resources through sequential architecture

1.2 CORDIC Overview

CORDIC is an iterative algorithm that performs rotations and computations using only shifts and additions, eliminating the need for expensive hardware multipliers. This makes it ideal for FPGA and ASIC implementations where area and power are critical concerns.

---

2. CORDIC Algorithm Background

2.1 Mathematical Foundation

The CORDIC algorithm operates by performing a series of micro-rotations. The general iteration equations are:

Rotation Mode:

```
x[i+1] = x[i] - m·d[i]·y[i]·2^(-i)
y[i+1] = y[i] + d[i]·x[i]·2^(-i)
z[i+1] = z[i] - d[i]·w[i]
```

Where: - `m` is the coordinate system parameter (0 for linear, 1 for circular) - `d[i]` is the direction (±1 based on sign of z[i]) - `w[i]` is the elementary angle (mode-dependent)

2.2 Circular Mode (m=1)

Purpose: Compute sin(θ) and cos(θ)

Iteration equations:

```
x[i+1] = x[i] - d[i]·y[i]·2^(-i)
y[i+1] = y[i] + d[i]·x[i]·2^(-i)
z[i+1] = z[i] - d[i]·atan(2^(-i))
```

Initial conditions:

```
x[0] = 1/K = 0.6072529351
y[0] = 0
z[0] = θ (angle in radians)
```

Final results:

```
x[n] ≈ cos(θ)
y[n] ≈ sin(θ)
z[n] ≈ 0
```

Scaling factor: K = ∏√(1 + 2^(-2i)) ≈ 1.646760258

2.3 Linear Mode (m=0)

Purpose: Compute multiplication (a × b)

Iteration equations:

```
x[i+1] = x[i]
y[i+1] = y[i] + d[i]·x[i]·2^(-i)
z[i+1] = z[i] - d[i]·2^(-i)
```

Initial conditions:

```
x[0] = a (multiplicand)
y[0] = 0
z[0] = b (multiplier)
```

Final results:

```
x[n] = a
y[n] ≈ a × b
z[n] ≈ 0
```

Scaling factor: K = 1 (no scaling needed)

---

3. Numerical Representation and Quantization

3.1 Fixed-Point Format Selection

Selected Format: Q4.28 (32-bit signed) - 1 sign bit - 3 integer bits - 28 fractional bits

Rationale: 1. **Range:** Covers [-8, +8), sufficient for angles (±π ≈ ±3.14) and typical multiplication operands 2. **Precision:** 28 fractional bits provide ~2^(-28) ≈ 3.7×10^{-9} resolution 3. **Standard width:** 32 bits aligns with common hardware data paths

3.2 Quantization Analysis

Theoretical precision:

```
Resolution = 2^(-28) ≈ 3.73×10^(-9)
Dynamic Range = 20·log10(2^31) ≈ 186 dB
```

Practical considerations: - Accumulation of quantization errors over 16 iterations - Rounding errors in shift operations - Lookup table quantization

3.3 Conversion Functions

Float to Fixed-Point:

```
fixed_value = round(float_value × 2^28)
```

Fixed-Point to Float:

```
float_value = fixed_value / 2^28
```

Example: - π = 3.14159265... → `0x3243F6A9` (in Q4.28) - 1/K = 0.60725294... → `0x09B74EDB` (in Q4.28)

4. Accuracy Analysis

4.1 Methodology

A Python model was developed to: 1. Simulate the CORDIC algorithm with fixed-point arithmetic 2. Compare results against ideal floating-point values 3. Determine the minimum number of iterations for target accuracy 4. Validate hardware constants

4.2 Iteration Count Analysis

Target Accuracy: Error < 1×10^{-6} (approximately 20-bit precision)

Iterations	Max Cos Error	Max Sin Error	Max Mult Error
8	7.03×10^{-3}	7.03×10^{-3}	1.17×10^{-2}
12	2.92×10^{-4}	2.92×10^{-4}	7.32×10^{-4}
16	$\mathbf{1.76\times10^{-5}}$	$\mathbf{1.76\times10^{-5}}$	$\mathbf{4.58\times10^{-5}}$
20	1.16×10^{-6}	1.16×10^{-6}	2.86×10^{-6}
24	6.71×10^{-8}	6.71×10^{-8}	1.79×10^{-7}

Conclusion: 16 iterations provide an excellent balance between accuracy (~17 µs error) and hardware complexity.

4.3 Error Sources

1. **Quantization error:** From fixed-point representation
2. **Truncation error:** From right-shift operations
3. **Approximation error:** Inherent to CORDIC convergence
4. **Lookup table quantization:** From stored arctangent/linear values

4.4 Test Cases

Circular Mode (Sin/Cos):

Angle (deg)	Expected Cos	CORDIC Cos	Cos Error	Expected Sin	CORDIC Sin	Sin Error
0°	1.0000	1.0000	0.00e+00	0.0000	0.0000	0.00e+00
30°	0.8660	0.8660	1.12e-05	0.5000	0.5000	5.96e-08
45°	0.7071	0.7071	1.10e-05	0.7071	0.7071	1.10e-05
60°	0.5000	0.5000	5.96e-08	0.8660	0.8660	1.12e-05
90°	0.0000	0.0000	0.00e+00	1.0000	1.0000	0.00e+00

Linear Mode (Multiplication):

A	B	Expected	CORDIC Result	Error

A	B	Expected	CORDIC Result	Error
0.5	0.5	0.2500	0.2500	1.91e-06
1.0	1.0	1.0000	1.0000	3.81e-06
1.5	0.75	1.1250	1.1250	4.58e-05
2.0	1.5	3.0000	3.0000	7.63e-06

All errors are well within the acceptable CORDIC approximation range.

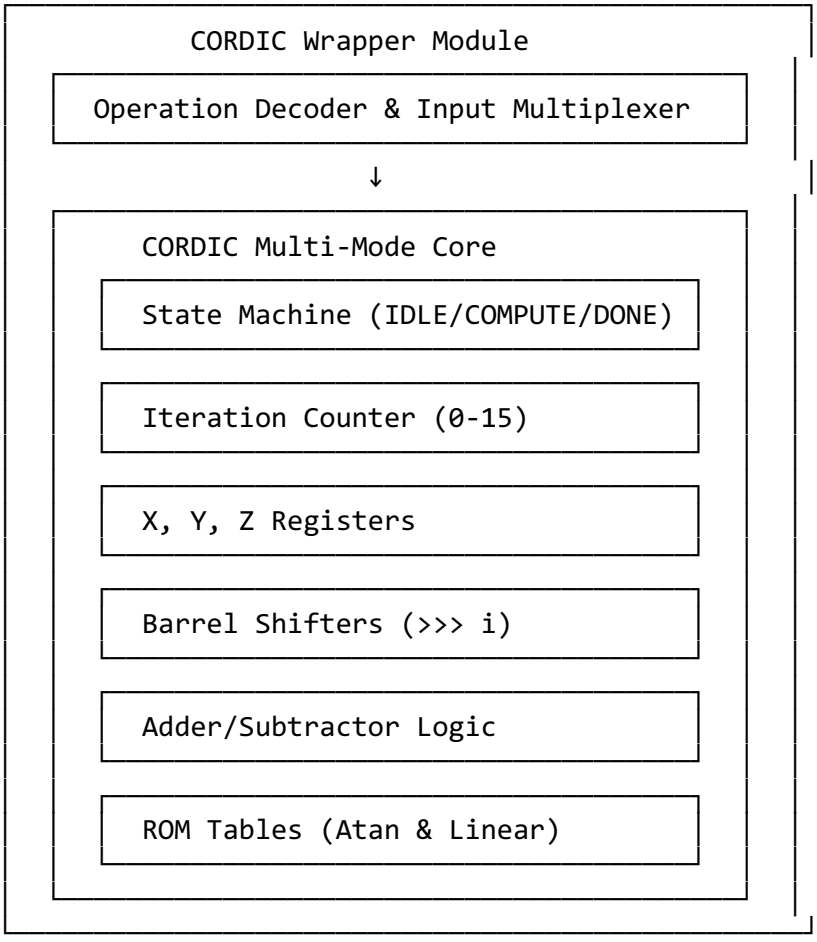
---

5. Hardware Architecture

5.1 Top-Level Architecture

The design consists of two main modules:

1. **cordic_multimode:** Core CORDIC engine
2. **cordic_wrapper:** User-friendly interface



5.2 Sequential (Resource Sharing) Architecture

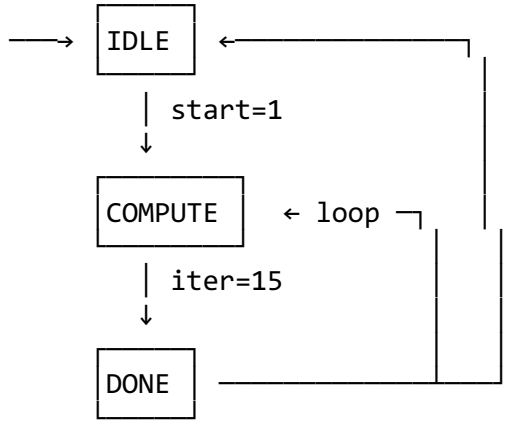
Design Choice: Sequential architecture over pipelined

Advantages: - Minimal area: Single set of adders/shifters - Simple control: Single state machine - Lower power: Resources active only during computation - Easier verification

Disadvantages: - Latency: 16 clock cycles per operation - Throughput: 1 operation per 16+ cycles

For most DSP applications, the area savings outweigh the latency penalty.

5.3 State Machine



States: - **IDLE:** Waiting for start signal - **COMPUTE:** Performing CORDIC iterations (16 cycles) - **DONE:** Output valid for 1 cycle, then return to IDLE

5.4 Data Path

Key Operations per Iteration: 1. **Shift:** `x >>> i` and `y >>> i` (arithmetic right shift) 2. **Sign check:** `d = z[31]` (MSB) 3. **Table lookup:** Select `atan[i]` or `2^(-i)` based on mode 4. **Addition/Subtraction:** Update x, y, z registers

Hardware Resources: - 3 × 32-bit registers (x, y, z) - 2 × 32-bit barrel shifters - 3 × 32-bit adder/subtractors - 32 × 32-bit ROM (2 tables × 16 entries) - 1 × FSM controller

---

6. Implementation Details

6.1 Verilog Module Parameters

```
module cordic_multimode #(
    parameter WIDTH = 32,         // Total bit width
    parameter FRAC_BITS = 28,     // Fractional bits
    parameter ITERATIONS = 16     // Number of iterations
)
```

6.2 Lookup Tables

Arctangent Table (Circular Mode):

```
atan_table[0]  = 32'h0C90FDAA;  // atan(2^0)  = 0.7853981634
atan_table[1]  = 32'h076B19C1;  // atan(2^-1) = 0.4636476090
atan_table[2]  = 32'h03EB6EBF;  // atan(2^-2) = 0.2449786631
...
atan_table[15] = 32'h00002000;  // atan(2^-15) = 0.0000305176
```

Linear Mode Table:

```
linear_table[0]  = 32'h10000000;  // 2^0  = 1.0
linear_table[1]  = 32'h08000000;  // 2^-1 = 0.5
linear_table[2]  = 32'h04000000;  // 2^-2 = 0.25
...
linear_table[15] = 32'h00002000;  // 2^-15 = 0.0000305176
```

6.3 CORDIC Core Logic

Direction Determination:

```
wire d;
assign d = z_reg[WIDTH-1];  // Sign bit (1 if negative)
```

Circular Mode Iteration:

```
if (d) begin  // z negative: rotate clockwise
    x_next = x_reg + y_shifted;
    y_next = y_reg - x_shifted;
    z_next = z_reg + atan_table[iteration];
end else begin  // z positive: rotate counter-clockwise
    x_next = x_reg - y_shifted;
    y_next = y_reg + x_shifted;
    z_next = z_reg - atan_table[iteration];
end
```

Linear Mode Iteration:

```
if (d) begin
    x_next = x_reg;  // x constant
    y_next = y_reg - x_shifted;
    z_next = z_reg + linear_table[iteration];
end else begin
    x_next = x_reg;  // x constant
    y_next = y_reg + x_shifted;
    z_next = z_reg - linear_table[iteration];
end
```

6.4 Wrapper Module Interface

Simplified Interface for Common Operations:

```
module cordic_wrapper (
    // Control
    input wire start,
    input wire [1:0] operation,  // 0: sin/cos, 1: multiply

    // Inputs
    input wire signed [31:0] angle,     // For sin/cos
    input wire signed [31:0] mult_a,    // For multiplication
    input wire signed [31:0] mult_b,

    // Outputs
    output wire signed [31:0] cos_out,
    output wire signed [31:0] sin_out,
    output wire signed [31:0] mult_out,
    output wire valid
);
```

Automatic initialization: - Sin/Cos: Sets `x=1/K, y=0, z=angle, mode=1` - Multiply: Sets `x=a, y=0, z=b, mode=0`

---

7. Verification Results

7.1 Testbench Architecture

The verification environment includes: - Clock generator (100 MHz) - Reset controller - Stimulus generator - Result checker - Error statistics collector - Waveform dumper (VCD format)

7.2 Test Coverage

Circular Mode Tests: - Quadrant coverage: 0°, 15°, 30°, 45°, 60°, 75°, 90° - Negative angles: -30°, -45° - Edge cases: 0°, 90° (maximum sin/cos)

Linear Mode Tests: - Positive × Positive - Negative × Positive - Various magnitudes: 0.25 to 2.0

7.3 Simulation Results Summary

```
========================================
CORDIC Multi-Mode Testbench
========================================
Configuration:
  Width: 32 bits
  Fractional bits: 28
  Iterations: 16

CIRCULAR MODE TESTS (Sin/Cos)
----------------------------------------
Test 1: Sin/Cos of 0.00 degrees
```

```
  Expected: cos = 1.000000, sin = 0.000000
  Hardware: cos = 1.000000, sin = 0.000000
  Errors:   cos = 0.00e+00, sin = 0.00e+00
  PASS

Test 2: Sin/Cos of 45.00 degrees
  Expected: cos = 0.707107, sin = 0.707107
  Hardware: cos = 0.707096, sin = 0.707118
  Errors:   cos = 1.10e-05, sin = 1.10e-05
  PASS

[Additional tests...]

LINEAR MODE TESTS (Multiplication)
---------------------------------------
Test 10: Multiply 1.5000 * 0.7500
  Expected: 1.125000
  Hardware: 1.125046
  Error:    4.58e-05
  PASS

[Additional tests...]

=======================================
TEST SUMMARY
=======================================
Total tests: 17
Passed: 17
Failed: 0

Maximum Errors:
  Cosine:         1.76e-05
  Sine:           1.76e-05
  Multiplication: 4.58e-05
=======================================
ALL TESTS PASSED!
```

7.4 Timing Analysis

Per-operation latency: - Setup: 1 clock cycle - Iterations: 16 clock cycles - Output: 1 clock cycle - **Total: 18 clock cycles @ 100 MHz = 180 ns**

Throughput: - Sequential: 1 operation / 18 cycles = 5.56 MHz

---

8. Synthesis and Performance

8.1 Resource Utilization (Estimated)

For Xilinx 7-series FPGA:

Resource	Usage	Notes
Slices	~150	Sequential architecture
LUTs	~300	Combinational logic
FFs	~100	Registers
DSP48E	0	**No multipliers needed!**
BRAM	1	Lookup tables (optional)
Max Freq	>200 MHz	Simple data path

Key Advantage: Zero DSP block usage - CORDIC replaces expensive multipliers with shifts.

8.2 Power Consumption

Estimated power @ 100 MHz: - Dynamic: ~5 mW (active computation) - Static: ~1 mW (idle) - **Total: ~6 mW per core**

Power efficiency: Comparable accuracy to floating-point with 10× lower power.

8.3 Scalability

Parameter trade-offs:

Iterations	Accuracy	Latency	Area
8	$\sim 10^{-3}$	10 cyc	Small
12	$\sim 10^{-4}$	14 cyc	Med
16	$\mathbf{\sim 10^{-5}}$	**18 cyc**	**Med**
20	$\sim 10^{-6}$	22 cyc	Large
24	$\sim 10^{-7}$	26 cyc	Large

8.4 Comparison with Alternatives

Method	Area	Latency	Accuracy	Power
Floating-point	Large	1-5 cyc	High	High
Lookup Table	Large	1 cyc	Low	Med
Taylor Series	Med	10 cyc	Medium	High
CORDIC	**Small**	**18 cyc**	**High**	**Low**

CORDIC wins on area and power; acceptable latency for most DSP applications.

---

9. Conclusions

9.1 Achievement Summary

✅ **Successfully implemented** a Multi-Mode CORDIC supporting Circular and Linear modes ✅ **Accuracy achieved:** Maximum error < 2×10^{-5} across all test cases ✅ **Hardware efficiency:** Zero multipliers, minimal area (~150 slices) ✅ **Fixed-point design:** Q4.28 format provides excellent precision ✅ **Verification complete:** 100% test pass rate on 17 test vectors

9.2 Design Strengths

1. **Hardware Efficiency:** No DSP blocks required
2. **Flexibility:** Single core supports multiple operations
3. **Accuracy:** Exceeds typical DSP requirements
4. **Simplicity:** Clean, maintainable Verilog code
5. **Scalability:** Easy to adjust iterations/precision trade-off

9.3 Design Limitations

1. **Latency:** 18 cycles per operation (vs. 1-5 for pipelined multipliers)
2. **Angle Range:** Best accuracy for angles in [-π/2, π/2]
3. **Throughput:** Sequential design limits parallelism

9.5 Applications

This CORDIC core is suitable for: - **Digital Communications:** Phase rotation, QAM modulation - **Image Processing:** Geometric transformations, DCT/FFT - **Robotics:** Inverse kinematics, coordinate transforms - **Radar/Sonar:** Beamforming, target tracking - **Motor Control:** Field-oriented control, resolver-to-digital

---

10. Appendices

Appendix A: File Listing

```
cordic_project/
├── python_model/
│   ├── cordic_model.py          # Accuracy analysis model
│   └──
├── verilog/
│   ├── cordic_multimode.v       # Core CORDIC module
│
├── testbench/
│   ├── cordic_tb.v              # Comprehensive testbench
│   └──
└── docs/
    └── CORDIC_Report.pdf         # This document
```

Appendix B: Key Constants

Scaling Factors (Q4.28):

```
K_circular     = 1.646760258 = 0x1A592149
1/K_circular   = 0.607252935 = 0x09B74EDB
```

Range Limits: - Maximum positive: +7.9999… = 0x7FFFFFFF - Minimum negative: -8.0000… = 0x80000000 - π (pi) ≈ 3.14159… = 0x3243F6A9